

Test code for IRQ and mutual exclusion between IRQ and main with LW_inc

Format: ADD RD, RS, RT
LW_inc RT, xINM(RS)
LW RT, INM(RS)
SW RT, INM(RS)
BEQ RS, RT

Data Memory: [256, 1, 8, 0, 12, 0xAB0, 0, 0x0BAD0C0D, Ticket_number, Turn, Ticket_IRQ...]

Pseudo-código

```
const int sp_init = 256;
const int one = 1;
const int eight = 8;
volatile static int Ticket_number = 1;
volatile static int Turn = 0;
volatile static int Ticket_IRQ = 0;

void main(){
    register int R1, R2;
    while(1) {
        //lock
        R1 = FetchAndIncrement(&Ticket_number);
        while(1) {
            R2 = Turn; //Wait for his turn
            IF (R1 = R2) {
                Print(8) // 8 is the code id of the main function
                Print(R1); //we want to verify that the turn
                management is correct
                R1 = FetchAndIncrement(&Turn); // It does not need
                to be atomic
                Print(R1);
            } //if
        } //while
    } //while
} //main

void RTI (void) __irq { //__irq indicates that it is an irq exception in order to generate to
generate the appropriate code
    register int R1, R2;
    R1 = Ticket_IRQ;
    R2 = Turn;
    IF (R1 = R2) {
        Print(256) // 256 is the code id of the IRQ
        Print(R1); //we want to verify that the turn management is
        correct
        FetchAndIncrement(&Turn); // It does not need to be atomic
        // IRQ asks for a new turn
        Ticket_IRQ = FetchAndIncrement(&Ticket_number);
    }
}
```

```
void Abort (void) __abort{//__abort indicates that it is an abort exception in order to generate
to generate the appropriate code
while(1){};}
```

```
void UNDEF (void) __undef {//__undef indicates that it is an undef exception in order to
generate to generate the appropriate code
while(1){};}
```

Expected behaviour: IRQ has the first turn (turn=0), and Main the second. For a while, they have alternating turns until 2015ns. At that moment, IRQ is always 1 and Main cannot be executed. Hence, the turn system is blocked.

Output_data: 0,256,0,8,1,256,2,8,3,256,4

Reset	@0x0	10210003	beq R1, R1, INI;	Jump to INI (@16)
IRQ	@0x4	1021003E	beq R1, R1, RTI;	Jump to @64*4
DAbort	@0x8	1021005D	beq R1, R1, RT_Abort;	Jump to @96*4
UNDEF	@0xC	1021006C	beq R1, R1, RT_UNDEF;	Jump to @112*4
INI:	@0x10	081F0000	LW R31, x0(R0)	R31=Mem(0) = 256; R31 is the stack pointer (SP)
main		40010020	LW_inc R1, x20(R0)	R1=Mem(32); Mem(32)++; //R1=Ticket_number; 1 memory stall
turn		08020024	LW R2, x24(R0)	R2=Mem(36); //R2=Turn
		10220001	beq R1, R2, print	If R1=R2 goto print. Data hazard. 2 stalls.
		1021FFFFD	beq R1, R1, turn	Goto turn
print		08010008	LW R1, x8(R0)	R1=Mem(8) = 8;
		0C017004	sw r1, x7004(R0)	IO_output <= R1; print(8); lw-use.
		0C027004	sw r2, x7004(R0)	IO_output <= R2; 1,2,3,4,...print= current turn
		40010024	LW_inc R1, x24(R0)	R1=Mem(36); Mem(36)++; //R1=Turn; 1 memory stall
		1021FFFF7	beq R1, R1, main	Goto main; Data hazard. 2 stalls.
RTI:	@0x100	0FE10000	SW R1, x0(R31)	Stores R1 in the stack
		0FE20004	SW R2, x4(R31)	Stores R2 in the stack
		08010008	LW R1, x8(R0)	R1=Mem(8) = 8;
		07E1F800	ADD r31, R1, R31	R31=R31 +8; Increment SP. lw-use
		08010028	LW R1, x28(R0)	R1=Mem(40); R1= Ticket_IRQ
		08020024	LW R2, x24(R0)	R2=Mem(36); R2 = Turn
		10220001	beq R1, R2, print	If R1=R2 goto IRQ_print. Data hazard. 2 stops
		10210006	beq R1, R1, IRQ_end	Goto IRQ_end
IRQ_print		08010000	LW R1, x0(R0)	R1=Mem(0) = 256;
		0C017004	sw r1, x7004(R0)	IO_output <= R1; print(256); lw-use.
		0C027004	sw r2, x7004(R0)	IO_output <= R2; 1,2,3,4,...print= current turn
		40010020	LW_inc R1, x20(R0)	R1=Mem(32); Mem(32)++; //R1=Ticket_number
		0C010028	SW R1, x28(R0)	Mem(40) = R1; // Ticket_IRQ= Ticket_number. Memory stall. Lw-use
		40010024	LW_inc R1, x24(R0)	R1=Mem(36); Mem(36)++; //R1=Turn; Memory stall
IRQ_end		08010004	LW R1, x4(R0)	R1=Mem(4) = 1;No lw-use!
		0CC17008	sw r1, x7008(r6)	INT_ACK <=1; (last one cycle). lw-use
		08010008	LW R1, x8(R0)	R1=Mem(8) = 8;
		07E1F801	SUB r31, R31, R1	SP=SP-8. lw-use
		0BE10000	LW R1, x0(R31)	R1 restored from stack
		0BE20004	LW R2, x4(R31)	R2 restored from stack
		20000000	Rte	Return to interrupted instruction
		08C17FFF	LW R1, x24000(R6)	Abort: out of range. It should not be executed
RT_Abort	@0x180	08C10014	LW R1, x20(R6)	R1=Mem(20) =0x00000AB0;
		0CC17004	sw r1, x7004(r6)	IO_output <=0x00000AB0;
	bucleA	1000FFFF	beq R0, R0, RT_Abort	Infinite loop
RT_UNDEF	@0x1C0	08C1001C	LW R1, x24(R6)	R1=Mem(28) = 0x0BAD0C0D;
		0CC17004	sw r1, x7004(r6)	IO_output <=0x00000AB0;
	bucleU	1000FFFF	beq R0, R0, RT_UNDEF	Infinite loop